

Universidad Técnica Federico Santa María
Campus Santiago San Joaquín
Bases de Datos Avanzadas

Tarea 2: Apache Cassandra

Informe

Nombre: Sebastián Andrés Santander Martínez
Rol: 202373608-2
Nombre: Jaime Guzmán Muñoz
Rol: 202373524-8

Segundo semestre 2026

Índice

1. Introducción	3
2. Preparación del entorno	3
2.1. Clúster de Cassandra	3
2.2. Keyspace	3
3. Preparación de los datos y diseño de las tablas	4
3.1. Diseño orientado a consultas	4
3.2. Generación y carga de datos	5
4. Parte 1: Consultas	6
4.1. Búsqueda por clave de partición	6
4.2. Consulta por nacionalidad	7
4.3. Consulta de personajes	7
4.4. LIMIT y ordenamiento	8
5. Parte 2: Procesamiento de consultas y diseño	9
5.1. TRACING	9
5.2. Consulta sin clave apropiada	11
5.3. Storage-Attached Indexing (SAI)	12
5.4. Desnormalización frente a índices	15
5.5. Escalabilidad	15
6. Parte 3: Comparación, alta disponibilidad y actualización	16
6.1. Comparación	16
6.1.1. Documentos embebidos vs Tablas desnormalizadas	16
6.1.2. Flexibilidad de esquema	17
6.1.3. Claves de partición y clustering	17
6.1.4. Índices	17
6.1.5. Consultas no previstas	17
6.1.6. Duplicación de información	17
6.1.7. Escalabilidad horizontal	18
6.1.8. Por qué el diseño de MongoDB no es apropiado para Casandra y su relación con las consultas	18
6.2. Alta disponibilidad	18
6.2.1. Explicación de los resultados y cálculo de Quórum	21

6.3.	Recuperación de nodos	21
6.3.1.	¿Qué es el <i>hinted handoff</i> ?	22
6.3.2.	Escrituras dirigidas a una réplica temporalmente inaccesible	22
6.3.3.	Función del proceso de <i>repair</i>	23
6.3.4.	Relación entre replicación y tolerancia a fallos	23
6.4.	Actualización y datos desnormalizados	23
6.4.1.	Por qué la desnormalización dificulta las actualizaciones	24
6.4.2.	Qué ventaja entrega	24
6.4.3.	Uso de BATCH	24
7.	Parte 4: Cierre	24
8.	Referencias y material complementario	25

1. Introducción

En este informe se repite el caso de Voice Managers de la tarea anterior, pero ahora sobre Apache Cassandra. La diferencia principal con MongoDB es que en Cassandra las tablas se diseñan a partir de las consultas que va a hacer la aplicación.

Se trabajó con un clúster de tres nodos levantado con Docker, un keyspace con factor de replicación 3 y tres tablas pensadas para las consultas pedidas. Sobre eso se analizaron las consultas con TRACING, se probó ALLOW FILTERING, se crearon índices SAI y se comparó la desnormalización con el uso de índices.

Los archivos de la tarea (`schema.cql`, el script de generación de datos, las consultas y las capturas) están en el repositorio público <https://github.com/Sxbayb/Tareas-BDDA>.

2. Preparación del entorno

2.1. Clúster de Cassandra

El clúster se levantó con el `docker-compose.yml` entregado en Aula, que define tres contenedores con Cassandra 5.0 (`cassandra1`, `cassandra2` y `cassandra3`).

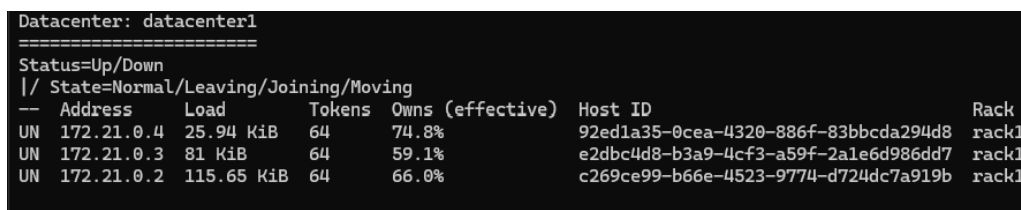
```
1 docker compose up -d
2 docker exec -it cassandra1 nodetool status
```

En el primer intento los tres contenedores se detuvieron a los pocos segundos con código de salida 137, que corresponde a un proceso terminado por falta de memoria. El log mostraba que Cassandra partía con 512 MB de heap y 256 MB de memoria directa, pero a eso se suma la memoria que usa la JVM, código compilado y más, con lo que el límite de 1 GB por contenedor no alcanzaba. Se subió `mem_limit` a 1536m en los tres nodos, sin tocar el heap, y se levantaron de a uno esperando que cada nodo quedara en estado UN antes de iniciar el siguiente.



```
[*] up 15/15
✓Image cassandra:5.0 Pulled 9.5s
✓Network tareas_cassandra_net Created 0.1s
✓Container cassandra3 Started 1.3s
✓Container cassandra1 Started 1.2s
✓Container cassandra2 Started 1.3s
```

Figura 1: Creación de la red y de los tres contenedores con Docker Compose.



```
Datacenter: datacenter1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address Load Tokens Owns (effective) Host ID Rack
UN 172.21.0.4 25.94 KiB 64 74.8% 92ed1a35-0cea-4320-886f-83bbcd294d8 rack1
UN 172.21.0.3 81 KiB 64 59.1% e2dbc4d8-b3a9-4cf3-a59f-2a1e6d986dd7 rack1
UN 172.21.0.2 115.65 KiB 64 66.0% c269ce99-b66e-4523-9774-d724dc7a919b rack1
```

Figura 2: Los tres nodos del clúster en estado UN (Up/Normal).

2.2. Keyspace

Antes de crear el keyspace se consultó el nombre del datacenter, que resultó ser `datacenter1`.

```
cqlsh:actors> SELECT data_center FROM system.local;

data_center
-----
datacenter1
```

Figura 3: Nombre del datacenter obtenido desde `system.local`.

El keyspace `actors` se creó con `NetworkTopologyStrategy` y factor de replicación 3. Como el clúster tiene tres nodos, cada nodo guarda una copia completa de los datos.

```
1 CREATE KEYSPACE IF NOT EXISTS actors
2 WITH replication = {
3     'class': 'NetworkTopologyStrategy',
4     'datacenter1': 3
5 };
```

```
cqlsh:actors> DESCRIBE KEYSPACE actors;

CREATE KEYSPACE actors WITH replication = {'class': 'NetworkTopologyStrategy', 'datacenter1': '3'} AND durable_writes = true;

CREATE TABLE actors.actors_by_id (
    actor_id uuid PRIMARY KEY,
    edad int,
    fecha_nacimiento date,
    nacionalidad text,
    nombre text,
    patrimonio bigint,
    idiomas set<text>
) WITH additional_write_policy = '99p'
AND allow_auto_snapshot = true
AND bloom_filter_fp_chance = 0.01
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND memtable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND incremental_backups = true
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';

CREATE TABLE actors.actors_by_nationality (
    nacionalidad text,
    edad int,
    actor_id uuid,
    fecha_nacimiento date,
    nombre text,
    patrimonio bigint,
    idiomas set<text>,
    PRIMARY KEY (nacionalidad, edad, actor_id)
```

Figura 4: Resultado de `DESCRIBE KEYSPACE actors` con la estrategia de replicación.

3. Preparación de los datos y diseño de las tablas

3.1. Diseño orientado a consultas

Las tablas se diseñaron a partir de las tres consultas mínimas del enunciado. Cada tabla resuelve una consulta usando solo su clave primaria.

Tabla	PRIMARY KEY	Partición / clustering	Consulta que resuelve
actors_by_id	(actor_id)	Partición: actor_id. Sin clustering.	Obtener un actor por su identificador.
actors_by_nationality	((nacionalidad), edad, actor_id)	Partición: nacionalidad. Clustering: edad ASC y actor_id.	Actores de una nacionalidad ordenados por edad.
characters_by_actor	((actor_id), rol, apariciones, nombre_personaje)	Partición: actor_id. Clustering: rol, apariciones DESC y nombre_personaje.	Personajes de un actor filtrando por rol y ordenados por apariciones.

Cuadro 1: Tablas del keyspace actors.

En `actors_by_nationality` se agregó `actor_id` al final del clustering porque dos actores del mismo país pueden tener la misma edad. Sin esa columna la clave sería la misma y el segundo INSERT sobrescribiría al primero, ya que en Cassandra un INSERT con una clave existente funciona como una actualización. Por la misma razón `nombre_personaje` cierra la clave de `characters_by_actor`.

En `characters_by_actor` el orden de las columnas de clustering importa. `rol` va primero para poder filtrar por igualdad (`rol = 'Principal'`) y `apariciones` va después con orden descendente, así que dentro de cada rol los personajes quedan guardados de mayor a menor. La columna `nombre_actor` se declaró `STATIC`, por lo que se guarda una sola vez por partición y no se repite en cada personaje.

Hay información duplicada. Nombre, fecha de nacimiento, patrimonio e idiomas están tanto en `actors_by_id` como en `actors_by_nationality`. Esa duplicación es la que permite responder la consulta por nacionalidad leyendo una sola partición, a cambio de tener que actualizar ambas tablas cuando cambia alguno de esos datos.

Las tres tablas están en `schema.cql`. Como ejemplo, la tabla de personajes:

```

1 CREATE TABLE IF NOT EXISTS characters_by_actor (
2     actor_id          uuid,
3     rol               text,
4     apariciones       int,
5     nombre_personaje  text,
6     nombre_actor      text STATIC,
7     produccion        text,
8     tipo_produccion   text,
9     generos           set<text>,
10    PRIMARY KEY ((actor_id), rol, apariciones, nombre_personaje)
11 ) WITH CLUSTERING ORDER BY (rol ASC, apariciones DESC, nombre_personaje ASC);

```

3.2. Generación y carga de datos

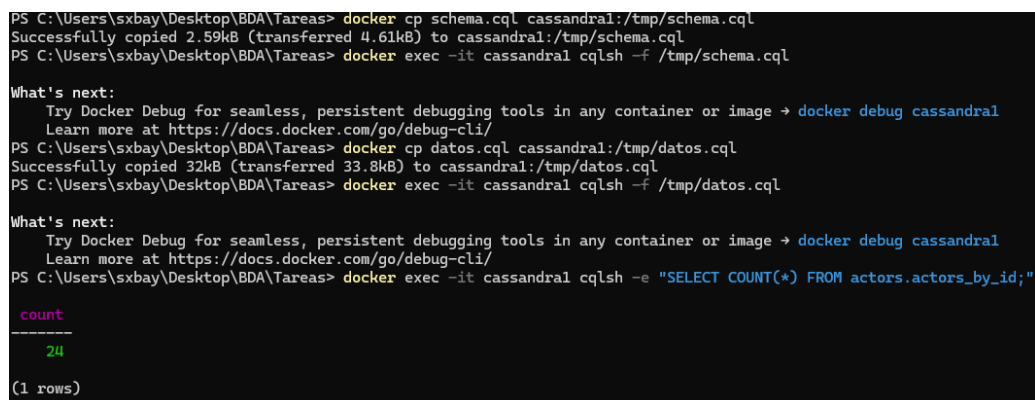
Los datos se generaron con `generar_datos.py`, que usa la biblioteca Faker con una semilla fija. El script crea 24 actores ficticios con identificador, nombre, nacionalidad, fecha de nacimiento, edad, patrimonio, idiomas (`set<text>`) y entre uno y siete personajes, cada uno con nombre, producción, tipo, rol, apariciones y géneros.

Algunos valores se fijaron a propósito para que las consultas tuvieran resultados útiles.

El identificador es un UUID y en Cassandra no hay autoincremento, porque en un sistema distribuido ningún nodo lleva la cuenta global de los ids.

El script escribe directamente `datos.cql` con los INSERT de las tres tablas. Cada actor se inserta dos veces (en `actors_by_id` y en `actors_by_nationality`) y cada personaje es una fila de `characters_by_actor`, en total 112 INSERT. Los archivos se copiaron al contenedor y se ejecutaron con `cqlsh -f`.

```
1 docker cp schema.cql cassandra1:/tmp/schema.cql
2 docker exec -it cassandra1 cqlsh -f /tmp/schema.cql
3 docker cp datos.cql cassandra1:/tmp/datos.cql
4 docker exec -it cassandra1 cqlsh -f /tmp/datos.cql
```



```
PS C:\Users\sxbay\Desktop\BDA\Tareas> docker cp schema.cql cassandra1:/tmp/schema.cql
Successfully copied 2.59kB (transferred 4.61kB) to cassandra1:/tmp/schema.cql
PS C:\Users\sxbay\Desktop\BDA\Tareas> docker exec -it cassandra1 cqlsh -f /tmp/schema.cql

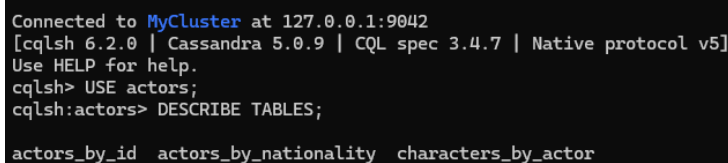
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug cassandra1
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\sxbay\Desktop\BDA\Tareas> docker cp datos.cql cassandra1:/tmp/datos.cql
Successfully copied 32kB (transferred 33.8kB) to cassandra1:/tmp/datos.cql
PS C:\Users\sxbay\Desktop\BDA\Tareas> docker exec -it cassandra1 cqlsh -f /tmp/datos.cql

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug cassandra1
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\sxbay\Desktop\BDA\Tareas> docker exec -it cassandra1 cqlsh -e "SELECT COUNT(*) FROM actors.actors_by_id;"

count
-----
      24
(1 rows)
```

Figura 5: Carga del esquema y de los datos, y conteo de los 24 actores.

El `COUNT(*)` sin clave de partición genera advertencia, porque para contar tiene que recorrer todas las particiones. Con 24 filas no importa, pero es el mismo problema que aparece más adelante con `ALLOW FILTERING`.



```
Connected to MyCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.9 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> USE actors;
cqlsh:actors> DESCRIBE TABLES;

actors_by_id actors_by_nationality characters_by_actor
```

Figura 6: Ingreso a `cqlsh` y tablas creadas en el keyspace.

4. Parte 1: Consultas

En todas las consultas se usó como ejemplo a la actriz Josefa Brenda Alarcón Álvarez, con `actor_id` `8c53765f-4ec0-4954-bf8b-2a6aab74fe57`.

4.1. Búsqueda por clave de partición

```
1 SELECT nombre, nacionalidad, edad, idiomas
2 FROM actors_by_id
3 WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57;
```

```
cqlsh:actors> SELECT nombre, nacionalidad, edad, idiomas
... FROM actors_by_id
... WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57;
```

nombre	nacionalidad	edad	idiomas
Josefa Brenda Alarcón Álvarez	Chile	23	{'Español', 'Italiano', 'Japones'}

(1 rows)

Figura 7: Actor obtenido por su identificador.

Explicación: `actor_id` es la clave de partición de la tabla. Cassandra aplica la función hash sobre el id, obtiene un token y con eso sabe exactamente qué nodos tienen esa partición. No necesita revisar ninguna otra partición, así que el costo de la consulta no depende de cuántos actores haya en la tabla. Los idiomas aparecen en orden alfabético porque un `set` se guarda ordenado.

4.2. Consulta por nacionalidad

```
1 SELECT nombre, edad, nacionalidad
2 FROM actors_by_nationality
3 WHERE nacionalidad = 'Chile'
4 LIMIT 10;
```

```
cqlsh:actors> SELECT nombre, edad, nacionalidad
... FROM actors_by_nationality
... WHERE nacionalidad = 'Chile'
... LIMIT 10;
```

nombre	edad	nacionalidad
Josefa Brenda Alarcón Álvarez	23	Chile
Andy Castro Aliaga	27	Chile
Rosa Angélica Uribe Álvarez	31	Chile
María Julieta Hernández Calderón	34	Chile
Catalina Camila Cáceres Dinamarca	38	Chile
Mia Ana Alarcón Rodríguez	39	Chile
María Patricia González Silva	41	Chile
Julio Pierre Tapia Gallardo	45	Chile
Uberlinda Lizana	52	Chile
Daniel Joaquín Villar Mena	58	Chile

(10 rows)

Figura 8: Actores chilenos ordenados por edad.

Explicación: `nacionalidad` es la clave de partición, por lo que todos los actores chilenos están juntos en la misma partición. `edad` es la primera columna de clustering, y las filas ya están guardadas en disco ordenadas por edad. Por eso el resultado sale ordenado sin escribir `ORDER BY`. El `LIMIT 10` deja fuera a dos chilenos.

4.3. Consulta de personajes

```
1 SELECT nombre_actor, nombre_personaje, rol, apariciones, produccion
2 FROM characters_by_actor
3 WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57
4 AND rol = 'Principal'
5 LIMIT 10;
```

```
cqlsh:actors> SELECT nombre_actor, nombre_personaje, rol, apariciones, produccion
... FROM characters_by_actor
... WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57
... AND rol = 'Principal'
... LIMIT 10;
```

nombre_actor	nombre_personaje	rol	apariciones	produccion
Josefa Brenda Alarcón Álvarez	José	Principal	93	Zaragoza 2125
Josefa Brenda Alarcón Álvarez	Alejandra	Principal	55	La Leyenda de Casandra
Josefa Brenda Alarcón Álvarez	Luís	Principal	4	Burgos 2189
Josefa Brenda Alarcón Álvarez	Alonso	Principal	2	Cronicas de Badajoz

(4 rows)

Figura 9: Personajes principales ordenados por apariciones.

Explicación: la consulta entrega la partición completa (`actor_id`) y una igualdad sobre la primera columna de clustering (`rol`). Dentro de la partición, las filas con rol Principal forman un bloque contiguo, ya ordenado por `apariciones` de mayor a menor (93, 55, 4 y 2) gracias al `CLUSTERING ORDER BY` definido al crear la tabla. La consulta se resuelve solo con la clave primaria.

4.4. LIMIT y ordenamiento

```
1 SELECT nombre, edad
2 FROM actors_by_nationality
3 WHERE nacionalidad = 'Chile'
4 ORDER BY edad DESC
5 LIMIT 5;
```

```
cqlsh:actors> SELECT nombre, edad
... FROM actors_by_nationality
... WHERE nacionalidad = 'Chile'
... ORDER BY edad DESC
... LIMIT 5;
```

nombre	edad
Alejandro Benavides-Gálvez Molina	70
Sergio Anderson Osorio Bastías	63
Daniel Joaquín Villar Mena	58
Uberlinda Lizana	52
Julio Pierre Tapia Gallardo	45

(5 rows)

Figura 10: Los cinco actores chilenos de mayor edad.

Para comprobar que no se puede ordenar por cualquier columna se intentó ordenar por patrimonio:

```
1 SELECT nombre, patrimonio
2 FROM actors_by_nationality
3 WHERE nacionalidad = 'Chile'
4 ORDER BY patrimonio DESC
5 LIMIT 5;
```

```
cqlsh:actors> SELECT nombre, patrimonio
... FROM actors_by_nationality
... WHERE nacionalidad = 'Chile'
... ORDER BY patrimonio DESC
... LIMIT 5;
InvalidRequest: Error from server: code=2200 [Invalid
query] message="Order by is currently only supported
on the clustered columns of the PRIMARY KEY, got pat
rimonio"
```

Figura 11: Error al ordenar por una columna que no es de clustering.

Explicación: Dentro de una partición Cassandra puede leer las filas en el orden del clustering o en el orden inverso. `ORDER BY edad DESC` solo invierte la lectura, así que Cassandra recorre la partición desde el final y se detiene en la quinta fila. `patrimonio` no es columna de clustering, y para ordenar por ella Cassandra tendría que leer toda la partición y ordenarla en memoria. Por eso responde *Order by is currently only supported on the clustered columns of the PRIMARY KEY*.

En una base de datos relacional se puede ordenar por cualquier columna porque trabaja sobre un solo servidor y aplica un sort al resultado. Cassandra evita ese trabajo a propósito, porque en una tabla distribuida con millones de filas un ordenamiento en memoria sería caro e impredecible. El orden se decide al diseñar la tabla, y si se necesita otro orden se crea otra tabla con un clustering distinto.

5. Parte 2: Procesamiento de consultas y diseño

Para leer los resultados de esta parte hay que saber qué dirección corresponde a cada nodo. `cqlsh` estaba conectado a `cassandra1`, por lo que ese nodo actuó como coordinador en todas las consultas.

Dirección	Nodo
172.21.0.2	cassandra1 (coordinador)
172.21.0.3	cassandra2
172.21.0.4	cassandra3

Cuadro 2: Direcciones de los nodos del clúster.

5.1. TRACING

Se activó `TRACING ON` y se repitieron las tres primeras consultas de la Parte 1.

```

cqlsh:actors> TRACING ON;
TRACING set to ON
cqlsh:actors> SELECT nombre, nacionalidad, edad, idiomas
... FROM actors_by_id
... WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57;

```

nombre	nacionalidad	edad	idiomas
Josefa Brenda Alarcón Álvarez	Chile	23	['Español', 'Italiano', 'Japones']

(1 rows)

Tracing session: 43181150-c2a1-11f1-b523-691d3fd4u10e

activity	timestamp	source	source_elapsed	client
Execute CQL query	2026-10-07 22:48:49.639000	172.21.0.2	0	127.0.0.1
Parsing SELECT nombre, nacionalidad, edad, idiomas FROM actors_by_id WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57;	2026-10-07 22:48:49.643000	172.21.0.2	5150	127.0.0.1
Preparing statement [Native-Transport-Requests-1]	2026-10-07 22:48:49.644000	172.21.0.2	5699	127.0.0.1
Reading data from /172.21.0.4:7000 message size 134 bytes [Messaging-EventLoop-3-0]	2026-10-07 22:48:49.644000	172.21.0.2	6438	127.0.0.1
Sending READ_REQ message to /172.21.0.4:7000 message size 134 bytes [Messaging-EventLoop-3-0]	2026-10-07 22:48:49.645000	172.21.0.2	6637	127.0.0.1
READ_REQ message received from /172.21.0.2:7000 [Messaging-EventLoop-3-9]	2026-10-07 22:48:49.649000	172.21.0.4	539	127.0.0.1
Speculating read retry on Full(cassandra/172.21.0.2:7000, (7887915888919699460,718388384972629561)) [Native-Transport-Requests-1]	2026-10-07 22:48:49.649000	172.21.0.2	16888	127.0.0.1
Sending READ_REQ message to cassandra/172.21.0.2:7000 [Native-Transport-Requests-1]	2026-10-07 22:48:49.649000	172.21.0.2	11184	127.0.0.1
READ_REQ message received from /172.21.0.2:7000 [Messaging-EventLoop-3-16]	2026-10-07 22:48:49.650000	172.21.0.2	11776	127.0.0.1
Executing single-partition query on actors_by_id [ReadStage-2]	2026-10-07 22:48:49.650001	172.21.0.2	12010	127.0.0.1
Acquiring stable references [ReadStage-2]	2026-10-07 22:48:49.650002	172.21.0.2	12149	127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-2]	2026-10-07 22:48:49.650003	172.21.0.2	12087	127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-2]	2026-10-07 22:48:49.651000	172.21.0.2	12816	127.0.0.1
Read 1 live rows and 0 tombstone cells [ReadStage-2]	2026-10-07 22:48:49.651001	172.21.0.2	12943	127.0.0.1
Enqueuing response to /172.21.0.2:7000 [ReadStage-2]	2026-10-07 22:48:49.651002	172.21.0.2	13021	127.0.0.1
Sending READ_RSP message to cassandra/172.21.0.2:7000 [Messaging-EventLoop-3-13]	2026-10-07 22:48:49.651003	172.21.0.2	13163	127.0.0.1
READ_RSP message received from /172.21.0.2:7000 [Messaging-EventLoop-3-16]	2026-10-07 22:48:49.652000	172.21.0.2	13584	127.0.0.1
Processing response from /172.21.0.2:7000 [RequestResponseStage-2]	2026-10-07 22:48:49.652001	172.21.0.2	13771	127.0.0.1
Executing single-partition query on actors_by_id [ReadStage-3]	2026-10-07 22:48:49.656000	172.21.0.4	7712	127.0.0.1
Acquiring stable references [ReadStage-3]	2026-10-07 22:48:49.656001	172.21.0.4	7882	127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-3]	2026-10-07 22:48:49.656002	172.21.0.4	8033	127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-3]	2026-10-07 22:48:49.657000	172.21.0.4	8521	127.0.0.1
Read 1 live rows and 0 tombstone cells [ReadStage-3]	2026-10-07 22:48:49.657001	172.21.0.4	8626	127.0.0.1
Enqueuing response to /172.21.0.2:7000 [ReadStage-3]	2026-10-07 22:48:49.657002	172.21.0.4	8666	127.0.0.1
Sending READ_RSP message to cassandra/172.21.0.2:7000 [Messaging-EventLoop-3-11]	2026-10-07 22:48:49.658000	172.21.0.4	8966	127.0.0.1
READ_RSP message received from /172.21.0.4:7000 [Messaging-EventLoop-3-11]	2026-10-07 22:48:49.659000	172.21.0.2	64	127.0.0.1
Processing response from /172.21.0.4:7000 [RequestResponseStage-3]	2026-10-07 22:48:49.659001	172.21.0.2	512	127.0.0.1
Request complete	2026-10-07 22:48:49.653423	172.21.0.2	10422	127.0.0.1

Figura 12: Tracing de la búsqueda por actor_id.

```

cqlsh:actors> SELECT nombre, edad, nacionalidad
... FROM actors_by_nationality
... WHERE nacionalidad = 'Chile'
... LIMIT 10;

```

nombre	edad	nacionalidad
Josefa Brenda Alarcón Álvarez	23	Chile
Andy Castro Aliaga	27	Chile
Rosa Angélica Uribe Álvarez	31	Chile
Maria Julieta Hernández Calderón	34	Chile
Catalina Camila Gómez Olguinacra	38	Chile
Mia Ana Alarcón Rodríguez	39	Chile
Maria Patricia González Silva	41	Chile
Julio Pierre Tapia Gallardo	45	Chile
Uberlinda Lizana	52	Chile
Daniel Joaquín Villalaz Mena	58	Chile

(10 rows)

Tracing session: c6177778-c2a1-11f1-b523-691d3fd4u10e

activity	timestamp	source	source_elapsed	client
Execute CQL query	2026-10-07 22:52:29.415000	172.21.0.2	0	127.0.0.1
Parsing SELECT nombre, edad, nacionalidad FROM actors_by_nationality WHERE nacionalidad = 'Chile' LIMIT 10;	2026-10-07 22:52:29.416000	172.21.0.2	479	127.0.0.1
Preparing statement [Native-Transport-Requests-1]	2026-10-07 22:52:29.416001	172.21.0.2	979	127.0.0.1
Reading data from /172.21.0.3:7000 [Native-Transport-Requests-1]	2026-10-07 22:52:29.417000	172.21.0.2	1632	127.0.0.1
Sending READ_REQ message to /172.21.0.3:7000 message size 135 bytes [Messaging-EventLoop-3-2]	2026-10-07 22:52:29.417001	172.21.0.2	1838	127.0.0.1
READ_REQ message received from /172.21.0.2:7000 [Messaging-EventLoop-3-5]	2026-10-07 22:52:29.420000	172.21.0.3	365	127.0.0.1
Speculating read retry on Full(172.21.0.4:7000, (-518681730192781851, -390382011148091030)) [Native-Transport-Requests-1]	2026-10-07 22:52:29.421000	172.21.0.2	5915	127.0.0.1
Sending READ_REQ message to /172.21.0.4:7000 [Native-Transport-Requests-1]	2026-10-07 22:52:29.421001	172.21.0.2	6355	127.0.0.1
READ_REQ message received from /172.21.0.2:7000 [Messaging-EventLoop-3-9]	2026-10-07 22:52:29.422000	172.21.0.4	118	127.0.0.1
Executing single-partition query on actors_by_nationality [ReadStage-4]	2026-10-07 22:52:29.423000	172.21.0.4	799	127.0.0.1
Acquiring stable references [ReadStage-4]	2026-10-07 22:52:29.423001	172.21.0.4	929	127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-4]	2026-10-07 22:52:29.423002	172.21.0.4	1226	127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-4]	2026-10-07 22:52:29.425000	172.21.0.4	3039	127.0.0.1
Read 10 live rows and 0 tombstone cells [ReadStage-4]	2026-10-07 22:52:29.425001	172.21.0.4	3152	127.0.0.1
Enqueuing response to /172.21.0.2:7000 [ReadStage-4]	2026-10-07 22:52:29.426000	172.21.0.4	3663	127.0.0.1
Sending READ_RSP message to cassandra/172.21.0.2:7000 [Messaging-EventLoop-3-1]	2026-10-07 22:52:29.427000	172.21.0.4	4060	127.0.0.1
READ_RSP message received from /172.21.0.3:7000 [Messaging-EventLoop-3-5]	2026-10-07 22:52:29.427000	172.21.0.2	11408	127.0.0.1
Processing response from /172.21.0.3:7000 [RequestResponseStage-4]	2026-10-07 22:52:29.427001	172.21.0.2	11726	127.0.0.1
READ_RSP message received from /172.21.0.4:7000 [Messaging-EventLoop-3-11]	2026-10-07 22:52:29.427002	172.21.0.2	12096	127.0.0.1
Processing response from /172.21.0.4:7000 [RequestResponseStage-4]	2026-10-07 22:52:29.427003	172.21.0.2	12373	127.0.0.1
Request complete	2026-10-07 22:52:29.427549	172.21.0.2	12549	127.0.0.1

Figura 13: Tracing de la consulta por nacionalidad en actors_by_nationality.

```

cqlsh:actors> SELECT nombre_actor, nombre_personaje, rol, apariciones, produccion
... FROM characters_by_actor
... WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57
... AND rol = 'Principal'
... LIMIT 10;

```

nombre_actor	nombre_personaje	rol	apariciones	produccion
Josefa Brenda Alarcón Álvarez	Joan	Principal	10	Zapatera 2125
Josefa Brenda Alarcón Álvarez	Alejandra	Principal	4	La Leyenda de Casanova
Josefa Brenda Alarcón Álvarez	Luis	Principal	4	Surge 2189
Josefa Brenda Alarcón Álvarez	Alonso	Principal	1	Crónicas de Rodrygo

(4 rows)

Tracing session: d098ecac-c2a1-11f1-b523-691d3fd4u10e

activity	timestamp	source	source_elapsed	client
Execute CQL query	2026-10-07 22:52:53.739000	172.21.0.2	0	127.0.0.1
Parsing SELECT nombre_actor, nombre_personaje, rol, apariciones, produccion FROM characters_by_actor WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57 AND rol = 'Principal' LIMIT 10;	2026-10-07 22:52:53.739000	172.21.0.2	611	127.0.0.1
Preparing statement [Native-Transport-Requests-1]	2026-10-07 22:52:53.739001	172.21.0.2	630	127.0.0.1
Executing single-partition query on characters_by_actor [ReadStage-2]	2026-10-07 22:52:53.740000	172.21.0.2	1078	127.0.0.1
Acquiring stable references [ReadStage-2]	2026-10-07 22:52:53.740001	172.21.0.2	1078	127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-2]	2026-10-07 22:52:53.740002	172.21.0.2	1101	127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-2]	2026-10-07 22:52:53.740003	172.21.0.2	1101	127.0.0.1
Read 3 live rows and 0 tombstone cells [ReadStage-2]	2026-10-07 22:52:53.741001	172.21.0.2	1101	127.0.0.1
Request complete	2026-10-07 22:52:53.741001	172.21.0.2	1101	127.0.0.1

Figura 14: Tracing de la consulta de personajes principales.

Consulta	Coordinador	Nodos que leyeron	Filas leídas	Tiempo
1.1 por actor_id	cassandra1	cassandra3 y cassandra1 (lectura especulativa)	1	14,4 ms
1.2 por nacionalidad	cassandra1	cassandra2 y cassandra3 (lectura especulativa)	10	12,5 ms
1.3 personajes	cassandra1	solo cassandra1 (lectura local)	5	3,6 ms

Cuadro 3: Resumen de los tracings de la Parte 1.

Nodo coordinador y nodos involucrados. En las tres consultas el coordinador fue cassandra1, el nodo al que estaba conectado cqlsh. El coordinador recibe la consulta, la parsea, calcula el token de la clave de partición y decide a qué réplica pedirle los datos. Como el keyspace tiene RF = 3 con tres nodos, cualquier nodo es réplica de cualquier partición.

En la consulta 1.1 el coordinador envió un `READ_REQ` a cassandra3. Poco después aparece *speculating read retry*: al no recibir respuesta dentro del tiempo esperado, el coordinador mandó la misma lectura a otra réplica, en este caso a sí mismo. Esto viene de la propiedad `speculative_retry = '99p'` de la tabla, visible en la Figura 4, y sirve para no quedar esperando a una réplica lenta. En la 1.2 pasó lo mismo con cassandra2 y cassandra3. En la 1.3 no hubo mensajes entre nodos, porque el coordinador leyó su propia copia.

Operaciones realizadas. Las tres consultas muestran *Executing single-partition query*, es decir, cada una leyó una sola partición. La línea *Read N live rows* indica cuántas filas se leyeron: 1 en la búsqueda por id, 10 en la consulta por nacionalidad (las primeras diez del orden del clustering) y 5 en la de personajes (los cuatro personajes principales más, probablemente, la fila estática con `nombre_actor`).

Las tres consultas terminaron entre 3,6 y 14,4 ms. La 1.3 fue la más rápida porque se resolvió de forma local, sin red. En las otras dos la mayor parte del tiempo se fue en el envío de mensajes entre nodos y en la lectura especulativa.

Las tres consultas entregan la clave de partición completa, por eso Cassandra sabe de antemano qué partición leer y no consulta rangos del anillo. El clustering se nota en que ninguna necesitó ordenar: la 1.2 leyó las diez primeras filas de la partición de Chile, que ya estaban ordenadas por edad, y la 1.3 leyó un bloque contiguo dentro de la partición del actor.

5.2. Consulta sin clave apropiada

Sobre la tabla principal se intentó filtrar por nacionalidad y edad, columnas que no forman parte de su clave primaria.

```
1 SELECT nombre, nacionalidad, edad
2 FROM actors_by_id
3 WHERE nacionalidad = 'Chile' AND edad > 40
4 LIMIT 10;
```

```
cqlsh:actors> TRACING OFF;
TRACING set to OFF
cqlsh:actors>
cqlsh:actors> SELECT nombre, nacionalidad, edad
... FROM actors_by_id
... WHERE nacionalidad = 'Chile' AND edad > 40
... LIMIT 10;
InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute this query as it might involve data filtering and thus may have unpredictable performance.
If you want to execute this query despite the performance unpredictability, use ALLOW FILTERING"
```

Figura 15: Cassandra rechaza la consulta sin clave de partición.

Cassandra respondió *Cannot execute this query as it might involve data filtering and thus may have unpredictable performance*. Luego se repitió la consulta agregando **ALLOW FILTERING** y con **TRACING ON**.

```
1 SELECT nombre, nacionalidad, edad
2 FROM actors_by_id
3 WHERE nacionalidad = 'Chile' AND edad > 40
4 LIMIT 10
5 ALLOW FILTERING;
```

The screenshot shows the Cassandra CLI interface. At the top, the query is executed with **ALLOW FILTERING**. Below the query, the results are displayed in a table format:

nombre	nacionalidad	edad
Julio Pierre Tapia Gallardo	Chile	45
Daniel Joaquín Villar Mena	Chile	58
Alejandro Benavides-Gálvez Molina	Chile	78
Uberlinda Lizana	Chile	52
Maria Patricia González Silva	Chile	41
Sergio Anderson Osorio Bastias	Chile	63

Below the results, the tracing output is shown, detailing the execution of the query. It includes information about the query plan, the execution of the **WHERE** clause, and the execution of the **SELECT** clause. The tracing output shows that the query was executed on 193 ranges, and the results were returned in a single batch.

Figura 16: Resultado y tracing con **ALLOW FILTERING**.

El tracing es muy distinto a los anteriores. Ya no aparece *single-partition query* sino *Submitting range requests on 193 ranges*, es decir, el coordinador tuvo que consultar todos los rangos de tokens del anillo. Luego aparece *Executing seq scan* sobre el rango que va desde el token mínimo hasta el mínimo, o sea la tabla completa, y *Read 24 live rows*. Cassandra leyó los 24 actores para devolver solo 6, y las otras 18 filas se leyeron y se descartaron. La consulta tardó 41,6 ms, entre tres y once veces más que las consultas por clave. Además el resultado sale desordenado, porque en esta tabla el orden lo da el token de cada **actor_id** y no la edad.

Con **RF = 3** un solo nodo tiene todos los rangos, por eso bastó una solicitud a **cassandra2**. En un clúster más grande, con datos repartidos, el coordinador tendría que preguntar a muchos nodos.

Por qué Cassandra restringe estas consultas. Sin clave de partición Cassandra no sabe en qué nodos están los datos que cumplen el filtro, así que la única opción es leer todo y filtrar después. El costo de la consulta pasa a depender del tamaño total de la tabla y no del tamaño del resultado. Con pocos datos no se nota, pero con grandes volúmenes un **ALLOW FILTERING** se convierte en un recorrido de todo el clúster que carga a todos los nodos, puede superar los timeouts de lectura y empeora a medida que la tabla crece. El **LIMIT** tampoco ayuda si las filas que cumplen la condición son pocas, porque Cassandra sigue leyendo hasta encontrarlas.

5.3. Storage-Attached Indexing (SAI)

Se crearon índices SAI sobre **nacionalidad** y **edad** en la tabla principal.

```
1 CREATE INDEX actors_nacionalidad_sai ON actors_by_id (nacionalidad) USING 'sai';
2 CREATE INDEX actors_edad_sai ON actors_by_id (edad) USING 'sai';
```

```
cqlsh:actors> TRACING OFF;
TRACING set to OFF
cqlsh:actors>
cqlsh:actors> CREATE INDEX actors_nacionalidad_sai ON actors_by_id (nacionalidad) USING 'sai';
cqlsh:actors> CREATE INDEX actors_edad_sai ON actors_by_id (edad) USING 'sai';
cqlsh:actors> TRACING OFF;
```

Figura 17: Creación de los índices SAI.

Con los índices creados se consultó por nacionalidad, por edad y por ambos atributos.

```
1 SELECT nombre, nacionalidad, edad FROM actors_by_id WHERE nacionalidad = 'Chile'
  LIMIT 10;
2 SELECT nombre, nacionalidad, edad FROM actors_by_id WHERE edad > 60 LIMIT 10;
3 SELECT nombre, nacionalidad, edad FROM actors_by_id
4 WHERE nacionalidad = 'Chile' AND edad > 40 LIMIT 10;
```

```
cqlsh:actors> SELECT nombre, nacionalidad, edad FROM actors_by_id WHERE nacionalidad = 'Chile' LIMIT 10;
```

nombre	nacionalidad	edad
Catalina Camila Cáceres Dinamarca	Chile	38
Julio Pierre Tapia Gallardo	Chile	45
María Julieta Hernández Calderón	Chile	34
Andy Castro Aliaga	Chile	27
Mía Ana Alarcón Rodríguez	Chile	39
Daniel Joaquín Villar Mena	Chile	58
Alejandro Benavides-Gálvez Molina	Chile	70
Uberlinda Lizana	Chile	52
María Patricia González Silva	Chile	41
Rosa Angélica Uribe Álvarez	Chile	31

(10 rows)

Figura 18: Consulta por nacionalidad usando SAI.

```
cqlsh:actors> SELECT nombre, nacionalidad, edad FROM actors_by_id WHERE edad > 60 LIMIT 10;
```

nombre	nacionalidad	edad
Alejandro Benavides-Gálvez Molina	Chile	70
William Sanchez	Estados Unidos	67
Kathryn Robinson	Estados Unidos	61
Sergio Anderson Osorio Bastías	Chile	63

(4 rows)

Figura 19: Consulta por rango de edad usando SAI.

```
cqlsh:actors> SELECT nombre, nacionalidad, edad FROM actors_by_id WHERE nacionalidad = 'Chile' AND edad > 40 LIMIT 10;
```

nombre	nacionalidad	edad
Julio Pierre Tapia Gallardo	Chile	45
Daniel Joaquín Villar Mena	Chile	58
Alejandro Benavides-Gálvez Molina	Chile	70
Uberlinda Lizana	Chile	52
María Patricia González Silva	Chile	41
Sergio Anderson Osorio Bastías	Chile	63

(6 rows)

Figura 20: Consulta combinando nacionalidad y edad.

La consulta combinada es la misma que en la sección anterior fue rechazada, y ahora se ejecuta sin `ALLOW FILTERING` con los mismos 6 resultados. La consulta por edad muestra que SAI también permite rangos (`>` y `<`) sobre columnas numéricas.

Después se creó un índice sobre los valores de la colección de idiomas y se buscó con `CONTAINS`.

```
1 CREATE INDEX actors_idiomas_sai ON actors_by_id (VALUES(idiomas)) USING 'sai';
2
3 SELECT nombre, nacionalidad, idiomas
4 FROM actors_by_id
5 WHERE idiomas CONTAINS 'Ingles'
6 LIMIT 10;
```

```
cqlsh:actors> CREATE INDEX actors_idiomas_sai ON actors_by_id (VALUES(idiomas)) USING 'sai';
cqlsh:actors>
cqlsh:actors> SELECT nombre, nacionalidad, idiomas FROM actors_by_id WHERE idiomas CONTAINS 'Ingles' LIMIT 10;
```

nombre	nacionalidad	idiomas
Christopher Braun	Estados Unidos	{'Ingles', 'Italiano'}
Julie Richardson	Estados Unidos	{'Ingles'}
Katherine Taylor	Estados Unidos	{'Ingles', 'Italiano', 'Japones'}
William Sanchez	Estados Unidos	{'Ingles', 'Japones', 'Portugues'}
Melanie Gutierrez	Estados Unidos	{'Ingles', 'Italiano'}
Kathryn Robinson	Estados Unidos	{'Frances', 'Ingles'}
Lorenzo Ignacio Gomez	Argentina	{'Español', 'Ingles'}
Terri Clark	Estados Unidos	{'Ingles'}

(8 rows)

Figura 21: Búsqueda de actores que hablan inglés con un índice sobre la colección.

Para comparar con la búsqueda por clave de partición se repitió la consulta por nacionalidad con **TRACING ON**.

```
cqlsh:actors> TRACING ON;
Tracing set to ON
cqlsh:actors> SELECT nombre, nacionalidad, edad FROM actors_by_id WHERE nacionalidad = 'Chile' LIMIT 10;
```

nombre	nacionalidad	edad
Catalina Camila Cáceres Dinamarca	Chile	38
Julio Pierre Tapia Gallardo	Chile	45
Maria Julieta Hernández Calderon	Chile	34
Andy Castro Altaga	Chile	27
Mia Ana Alarcón Rodríguez	Chile	39
Daniel Joaquín Villar Rera	Chile	58
Alejandro Benavides-Gálvez Molina	Chile	78
Uberlinda Lizana	Chile	82
Maria Patricia González Silva	Chile	41
Rosa Angélica Uribe Álvarez	Chile	31

(10 rows)

Tracing session: b0b22a80-c2a3-11f1-b523-691d3fd4410e

activity	timestamp	source	source_elapsed	client
Parsing SELECT nombre, nacionalidad, edad FROM actors_by_id WHERE nacionalidad = 'Chile' LIMIT 10;	2026-10-07 23:06:12.512000	172.21.0.2	0	127.0.0.1
Execute CQL3 query	2026-10-07 23:06:12.512001	172.21.0.2	367	127.0.0.1
Preparing statement	2026-10-07 23:06:12.512001	172.21.0.2	413	127.0.0.1
Index mean cardinalities are actors_nacionalidad_sai:9223372036854775807. Scanning with actors_nacionalidad_sai.	2026-10-07 23:06:12.515000	172.21.0.2	2126	127.0.0.1
Computing ranges to query	2026-10-07 23:06:12.515001	172.21.0.2	3288	127.0.0.1
Submitted 1 concurrent range requests	2026-10-07 23:06:12.516000	172.21.0.2	3476	127.0.0.1
Submitting range requests on 193 ranges with a concurrency of 100	2026-10-07 23:06:12.517000	172.21.0.2	5237	127.0.0.1
Executing read on actors.actors_by_id using index actors_nacionalidad_sai	2026-10-07 23:06:12.518000	172.21.0.2	3700	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.519000	172.21.0.2	6776	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.519001	172.21.0.2	6866	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.519002	172.21.0.2	7887	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.519003	172.21.0.2	7383	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.520000	172.21.0.2	8189	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.520001	172.21.0.2	8286	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.520002	172.21.0.2	8338	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.521000	172.21.0.2	8408	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.521001	172.21.0.2	8976	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.521002	172.21.0.2	9059	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.521003	172.21.0.2	9161	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.521004	172.21.0.2	9270	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.522000	172.21.0.2	9664	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.522001	172.21.0.2	9666	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.522002	172.21.0.2	9697	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.522003	172.21.0.2	9722	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.522004	172.21.0.2	9932	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.522005	172.21.0.2	9964	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.522006	172.21.0.2	9989	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.522007	172.21.0.2	10037	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.522008	172.21.0.2	10297	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.522009	172.21.0.2	10291	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.522010	172.21.0.2	10266	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.522011	172.21.0.2	10336	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.523000	172.21.0.2	11027	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.523001	172.21.0.2	11111	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.523002	172.21.0.2	11208	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.524000	172.21.0.2	11518	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.524001	172.21.0.2	11816	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.524002	172.21.0.2	11884	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.524003	172.21.0.2	11981	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.524004	172.21.0.2	12015	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.524005	172.21.0.2	12266	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.524006	172.21.0.2	12317	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.524007	172.21.0.2	12384	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.525000	172.21.0.2	12418	127.0.0.1
Executing single-partition query on actors_by_id	2026-10-07 23:06:12.525001	172.21.0.2	12673	127.0.0.1
Acquiring stable references	2026-10-07 23:06:12.525002	172.21.0.2	12725	127.0.0.1
Merging memtable contents	2026-10-07 23:06:12.525003	172.21.0.2	12784	127.0.0.1
Key cache hit for sstable 1, size = 0	2026-10-07 23:06:12.525004	172.21.0.2	12865	127.0.0.1
Read 10 live rows and 0 tombstone cells	2026-10-07 23:06:12.525005	172.21.0.2	13141	127.0.0.1
Index query accessed memtable indexes, 1 SSTable index, and 1 segment, post-filtered 10 rows in 10 partitions, and took 7556 microseconds.	2026-10-07 23:06:12.525006	172.21.0.2	13395	127.0.0.1
Request complete	2026-10-07 23:06:12.526010	172.21.0.2	14810	127.0.0.1

Figura 22: Tracing de la consulta por nacionalidad usando el índice SAI.

El tracing muestra *Scanning with actors_nacionalidad_sai*, así que el coordinador eligió el índice. Igual que con **ALLOW FILTERING**, aparece *Submitting range requests on 193 ranges*. Esto pasa porque los índices SAI son locales: cada nodo indexa solo los datos que guarda, y sin clave de partición hay que preguntarle al índice de todos los rangos. La diferencia está en lo que se lee después. El índice indica qué filas cumplen la condición y Cassandra ejecuta una *single-partition query* por cada una, diez en total, y termina con *post-filtered 10 rows in 10 partitions*. No leyó filas que no sirvieran. La consulta tardó 14,8 ms. En este tracing aparece también *Key cache hit for sstable 1*, porque al crear los índices Cassandra escribió a disco los datos que tenía en memoria.

Comparación con la búsqueda por clave de partición. Una consulta por clave va directo a una partición en las réplicas que corresponden. Una consulta por SAI tiene que consultar el índice en todo el anillo y luego hacer una lectura por cada partición encontrada. SAI evita el recorrido completo de **ALLOW FILTERING**, pero sigue siendo más cara que una consulta por clave, y la diferencia crece con la cantidad de nodos y de resultados.

Por qué no indexar todo. Cada índice SAI ocupa espacio en disco junto a cada sstable y se actualiza en cada escritura de la tabla, por lo que más índices significan escrituras más lentas y más trabajo durante las compactaciones. Los índices sobre columnas con pocos valores distintos, como nacionalidad, devuelven muchísimas filas cuando la tabla es grande y terminan consultando casi todas las particiones. Los índices sirven para consultas ocasionales o para filtrar dentro de una partición ya conocida, pero no reemplazan una buena clave de partición para las consultas frecuentes.

5.4. Desnormalización frente a índices

Para la misma consulta (actores chilenos) se compararon la tabla diseñada para ella (Figura 13) y la tabla principal con el índice SAI (Figura 22). Se agrega el caso con `ALLOW FILTERING` como referencia.

Forma de consulta	Qué consulta	Qué lee	Tiempo
<code>actors_by_nationality</code>	una partición (Chile)	10 filas contiguas, ya ordenadas por edad	12,5 ms
<code>actors_by_id</code> con SAI	los 193 rangos del anillo	10 particiones distintas, sin orden	14,8 ms
<code>actors_by_id</code> con <code>ALLOW FILTERING</code>	los 193 rangos del anillo	las 24 filas de la tabla	41,6 ms

Cuadro 4: Comparación de las formas de consultar por nacionalidad.

Con 24 actores los tiempos se parecen, así que la comparación importante es la cantidad de trabajo.

Uso de la clave de partición. En la tabla desnormalizada la nacionalidad es la clave de partición, así que la consulta va directo a una partición y además sale ordenada por edad gracias al clustering. Con SAI la nacionalidad es una columna común, el coordinador no sabe dónde están los datos y tiene que consultar todos los rangos. El resultado tampoco sale ordenado por edad, y no se le puede pedir `ORDER BY edad` porque la edad no es clustering en esa tabla. Y en cuanto a la **Duplicación de información.** La tabla desnormalizada repite nombre, fecha de nacimiento, patrimonio e idiomas de cada actor. SAI no duplica las filas, solo guarda la estructura del índice.

Con la tabla desnormalizada cada cambio en un dato duplicado se tiene que escribir en las dos tablas, y es la aplicación la que debe mantenerlas coherentes, porque Cassandra no lo hace, por lo que hay un **costo de mantención**. Un cambio de edad es más caro todavía: como la edad es parte de la clave de `actors_by_nationality`, no se puede actualizar, hay que borrar la fila y volver a insertarla. Con SAI la mantención la hace Cassandra, que actualiza el índice en cada escritura, pero eso agrega costo a todas las escrituras de la tabla.

Índices adicionales. Si se quisiera consultar por otras columnas habría que elegir entre crear más tablas desnormalizadas (más espacio y más escrituras por cada cambio) o más índices (consultas que recorren el anillo y escrituras más lentas). En general, para una consulta frecuente conviene la tabla diseñada para ella, y los índices quedan para consultas ocasionales.

5.5. Escalabilidad

Si la base tuviera 10 mil millones de registros el comportamiento de cada tipo de consulta cambiaría de la siguiente forma.

En cuanto a las **Consultas por clave de partición y clustering**. Siguen yendo directo a las réplicas de una partición, así que su costo depende del tamaño de esa partición y no del total de la tabla. La búsqueda por `actor_id` se mantendría prácticamente igual. El problema aparece en `actors_by_nationality`: con 10 mil millones de actores la partición de un país podría tener cientos de millones de filas, todas en los mismos tres nodos. Eso genera una partición demasiado grande y un punto caliente en el clúster. Se resolvería con una clave de partición compuesta, por ejemplo `((nacionalidad, rango_edad), edad, actor_id)`, que reparte un mismo país entre varias particiones más pequeñas.

Si se quisieran hacer **consultas con ALLOW FILTERING**. Tendrían que leer los 10 mil millones de filas para quedarse con las que cumplen la condición. El tiempo crecería con el total de datos, la consulta superaría los timeouts y afectaría a todos los nodos del clúster mientras se ejecuta. En la práctica no serían utilizables.

La **consultas mediante SAI** evitan leer filas que no cumplen el filtro, pero siguen consultando el índice en todos los nodos. Funcionan bien cuando el filtro es selectivo y devuelve pocas filas. Con un filtro poco selectivo, como una nacionalidad con millones de actores, el coordinador tendría que reunir resultados de todo el clúster y hacer una lectura por cada partición encontrada. Rinden mucho mejor si se combinan con la clave de partición, porque así el índice solo se consulta dentro de esa partición.

Cantidad de particiones o nodos consultados. Una consulta por clave de partición consulta solo las réplicas de esa partición, que con $RF = 3$ son tres nodos como máximo, y con consistencia ONE basta uno, sin importar cuántos nodos tenga el clúster. **ALLOW FILTERING** y **SAI** sin clave de partición consultan todos los rangos del anillo, así que si el clúster crece a cientos de nodos para guardar los datos, estas consultas tienen que hablar con todos ellos.

Y en **almacenamiento**. Con $RF = 3$ cada fila se guarda tres veces. La desnormalización multiplica eso: los datos de cada actor están en `actors_by_id` y en `actors_by_nationality`, así que con 10 mil millones de actores habría unas seis copias de esa información en el clúster. Cada índice SAI agrega su propio espacio en cada réplica. En Cassandra se acepta pagar más disco para que las lecturas sean simples, pero eso hay que tenerlo presente al dimensionar el clúster.

Por qué conocer las consultas antes. En Cassandra la clave primaria decide dónde se guardan los datos y en qué orden, y no se puede cambiar después de crear la tabla. Si aparece una consulta que no coincide con ninguna clave, las únicas salidas son **ALLOW FILTERING**, que no escala, un índice, que tiene costo en todo el clúster, o crear una tabla nueva y volver a cargar todos los datos. Por eso el diseño parte de la lista de consultas de la aplicación y no del modelo de entidades, al revés de lo que se hace en una base relacional.

6. Parte 3: Comparación, alta disponibilidad y actualización

6.1. Comparación

La diferencia existente entre los modelos de datos utilizados en estas dos tecnologías radica principalmente en las visiones que se implementan en cada una.

6.1.1. Documentos embebidos vs Tablas desnormalizadas

MongoDB utiliza un modelo jerárquico donde las entidades relacionadas se agrupan en un solo documento. Esto permite recuperar toda la información en una única lectura en disco. Por otro

lado, Cassandra para consultas complejas sigue una filosofía de *tablas desnormalizadas*. En lugar de anidar profundamente, en Cassandra se opta por crear múltiples tablas planas donde exista redundancia de datos según sea necesario con tal de satisfacer la lectura.

6.1.2. Flexibilidad de esquema

MongoDB posee un esquema dinámico. En esta tecnología es posible tener documentos con estructuras muy distintas entre sí, pudiendo agregar nuevos campos o insertar estructuras dentro de un documento sin afectar a los registros existentes. Cassandra tiene un esquema estático, cada columna y su tipo de dato deben estar estrictamente especificados, cualquier alteración estructural requiere modificar la estructura de la tabla general, o incluso requiriendo la creación de nuevas tablas, este tipo de esquema es muy similar al utilizado en bases de datos relacionales.

6.1.3. Claves de partición y clustering

En MongoDB se utiliza el `_id` como identificador único, pero la distribución de los datos a lo largo del clúster depende de la Shard Key que se encuentra desligada de este identificador. En Cassandra el modelo de datos depende enteramente de la Clave Primaria. La clave de partición determina en que nodo físico reside el dato, y la clave de clustering define el ordenamiento secuencial en el disco dentro de esta misma partición, logrando lecturas extremadamente rápidas.

6.1.4. Índices

Cassandra utiliza la clave de partición como índice primario, si bien admite índices secundarios no se recomienda su uso ya que estas son locales en cada nodo y su utilización puede requerir consultar en múltiples nodos o discos, generando problemas de rendimiento si se ocupan en columnas de baja cardinalidad. MongoDB ofrece índices secundarios altamente versátiles, permitiendo crear flujos de búsqueda mucho más personalizables y eficientes para algunos casos.

6.1.5. Consultas no previstas

Por la filosofía utilizada en Cassandra, una consulta no prevista es altamente ineficiente ya que esto implicaría no tener una tabla preparada para satisfacer la consulta, provocando un escaneo completo de la tabla, colapsando el rendimiento del clúster. MongoDB es excelente para este tipo de consultas, ya que al tener un motor de indexación flexible es posible responder de forma fácil a nuevas consultas sin tener que rediseñar la base de datos.

6.1.6. Duplicación de información

En MongoDB se fomenta la encapsulación mediante documentos embebidos para reducir la duplicación, acercándose a la normalización conceptual, aunque sin las restricciones relacionales. En Cassandra, la duplicación de datos es su principio fundacional. Esto se debe a que el espacio en disco es barato, mientras que la latencia de red es costosa, la misma información de un actor se escribe en tantas tablas diferentes como consultas existan para buscarlo.

6.1.7. Escalabilidad horizontal

Cassandra fue diseñada nativamente con una arquitectura *peer-to-peer*, lo que quiere decir que no existe un nodo maestro en la topología de Cassandra. Todos los nodos son iguales, lo que permite escalar de forma verdaderamente lineal y sin cuellos de botella simplemente añadiendo nodos al anillo. MongoDB escala horizontalmente con una arquitectura de sharding. Este tipo de arquitectura requiere de un proceso de balanceo de chunks que puede degradar la escalabilidad horizontal.

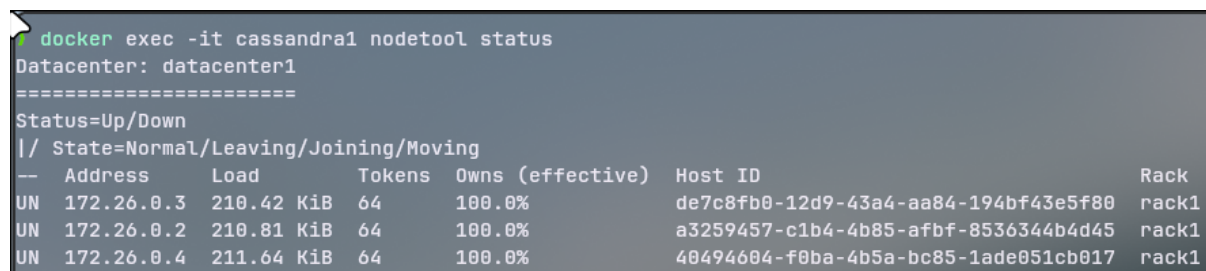
6.1.8. Por qué el diseño de MongoDB no es apropiado para Casandra y su relación con las consultas

Si se traslada el modelo jerárquico de MongoDB directamente a Cassandra, se genera una base de datos donde solo se puede buscar un actor si ya se conoce su identificador. Debido a la incapacidad de Cassandra para realizar agrupaciones dinámicas o búsquedas eficientes sobre campos no particionados sin afectar la latencia, este modelo basado en entidades colapsa ante requerimientos de filtrado.

En Cassandra, se utiliza un diseño dirigido por las consultas. La relación entre el diseño de las tablas y las consultas esperadas es absoluta: la cláusula `WHERE` de la consulta define la clave de partición. Por lo tanto, mientras en MongoDB se crea una colección de actores y se agregan índices, en Cassandra se diseñan tablas por cada consulta que se busque satisfacer, duplicando los datos para garantizar que cada lectura se dirija exactamente al nodo físico que posee la respuesta precalculada.

6.2. Alta disponibilidad

En primer lugar, se verifica la disponibilidad de los nodos.



```

docker exec -it cassandra1 nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens  Owns (effective)  Host ID                               Rack
UN  172.26.0.3    210.42 KiB    64       100.0%            de7c8fb0-12d9-43a4-aa84-194bf43e5f80  rack1
UN  172.26.0.2    210.81 KiB    64       100.0%            a3259457-c1b4-4b85-afbf-8536344b4d45  rack1
UN  172.26.0.4    211.64 KiB    64       100.0%            40494604-f0ba-4b5a-bc85-1ade051cb017  rack1

```

Figura 23: Comprobación del estado de los nodos del clúster.

Luego, se aplica un comando de inserción y un comando de consulta.

```
cqlsh> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh> INSERT INTO actors.characters_by_actor (actor_id, rol, apariciones, nombre_personaje, nombre_actor, produccion, tipo_produccion, generos) VALUES (8c53765f-4ec0-4954-bf8b-2a6aab74fe57, 'Principal', 31, 'Ma
ria', 'Josefa Brenda Alarcón Álvarez', 'El despido', 'Independiente', {'Fantasia', 'Terror'});
cqlsh> SELECT nombre_actor, nombre_personaje, rol, apariciones, produccion FROM actors.characters_by_actor WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab74fe57 AND rol = 'Principal' LIMIT 10;
```

nombre_actor	nombre_personaje	rol	apariciones	produccion
Josefa Brenda Alarcón Álvarez	José	Principal	93	Zaragoza 2125
Josefa Brenda Alarcón Álvarez	Alejandra	Principal	55	La Leyenda de Casandra
Josefa Brenda Alarcón Álvarez	Maria	Principal	31	El despido
Josefa Brenda Alarcón Álvarez	Luis	Principal	4	Burgos 2189
Josefa Brenda Alarcón Álvarez	Alonso	Principal	2	Cronicas de Badajoz

(5 rows)

Figura 24: Primer consistencia 1.

Tras esta primera ejecución se tuvo éxito en todas las operaciones. Ahora procedemos a bajar un nodo:

```
docker stop cassandra3
cassandra3
docker exec -it cassandra1 nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load        Tokens  Owns (effective)  Host ID                               Rack
UN 172.26.0.3    210.42 KiB   64      100.0%            de7c8fb0-12d9-43a4-aa84-194bf43e5f80 rack1
DN 172.26.0.2    210.81 KiB   64      100.0%            a3259457-c1b4-4b85-afbf-8536344b4d45 rack1
UN 172.26.0.4    211.64 KiB   64      100.0%            40494604-f0ba-4b5a-bc85-1ade051cb017 rack1
```

Figura 25: Nodo Cassandra3 down.

Una vez que está inactivo el nodo volvemos a aplicar consistencia de un nodo y las respectivas operaciones de inserción y consulta.

```
cqlsh> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh> INSERT INTO actors.actors_by_id (actor_id, nombre, nacionalidad, fecha_nacimiento, edad, patrimonio, idiomas) VALUES (8c53765f-4ec0-4954-bf8b-2a6aab75fe68, 'Jose Ignacio Álvarez', 'Chile', '2004-04-01', 2
2, 9557000, {'Español', 'Italiano'});
cqlsh> SELECT nombre FROM actors.actors_by_id WHERE actor_id = 8c53765f-4ec0-4954-bf8b-2a6aab75fe68;
```

nombre
Jose Ignacio Álvarez

(1 rows)

Figura 26: Segunda consistencia 1.

Se repite el proceso con **Consistency Quorum**;

```
cqlsh> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh> INSERT INTO actors.actors_by_id (actor_id, nombre, nacionalidad, fecha_nacimiento, edad, patrimonio, idiomas) VALUES (123e4567-e89b-12d3-a456-426614174001, 'Laura Gómez', 'España', '1995-10-15', 31, 4500
000, {'Español', 'Inglés'});
cqlsh> SELECT nombre FROM actors.actors_by_id WHERE actor_id = 123e4567-e89b-12d3-a456-426614174001;
```

nombre
Laura Gómez

(1 rows)

Figura 27: Primera consistencia Quorum.

Por último, se repite el proceso con **Consistency ALL**;

```
cqlsh> CONSISTENCY ALL;
Consistency level set to ALL.
cqlsh> INSERT INTO actors.actors_by_id (actor_id, nombre, nacionalidad, fecha_nacimiento, edad, patrimonio, idiomas) VALUES (223e4567-e89b-12d3-a456-426614174002, 'Kenji Sato', 'Japón', '1988-03-22', 38, 12500000, {'Japonés', 'Inglés'});
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level ALL" info={\'consistency\': \'ALL\', \'required_replicas\': 3, \'alive_replicas\': 2})})
cqlsh> SELECT nombre FROM actors.actors_by_id WHERE actor_id = 123e4567-e89b-12d3-a456-426614174001;
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level ALL" info={\'consistency\': \'ALL\', \'required_replicas\': 3, \'alive_replicas\': 2})})
```

Figura 28: Primera consistencia All.

Se puede observar como para este caso la consulta falló.

Ahora bajaremos un nodo más de la topología:

```
> docker stop cassandra2
cassandra2
> docker exec -it cassandra1 nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens  Owns (effective)  Host ID                               Rack
UN 172.26.0.3     210.42 KiB    64      100.0%            de7c8fb0-12d9-43a4-aa84-194bf43e5f80 rack1
DN 172.26.0.2     210.81 KiB    64      100.0%            a3259457-c1b4-4b85-afbf-8536344b4d45 rack1
DN 172.26.0.4     211.64 KiB    64      100.0%            40494604-f0ba-4b5a-bc85-1ade051cb017 rack1
```

Figura 29: Nodo Cassandra2 down.

Y aplicaremos **CONSISTENCY ONE**; repitiendo el proceso de inserción y consulta.

```
cqlsh> CONSISTENCY ONE;
Consistency level set to ONE.
cqlsh> INSERT INTO actors.actors_by_id (actor_id, nombre, nacionalidad, fecha_nacimiento, edad, patrimonio, idiomas) VALUES (323e4567-e89b-12d3-a456-426614174003, 'Claire Dubois', 'Francia', '2001-07-09', 25, 2100000, {'Francés', 'Alemán', 'Español'});
cqlsh> SELECT nombre FROM actors.actors_by_id WHERE actor_id = 323e4567-e89b-12d3-a456-426614174003;

nombre
-----
Claire Dubois

(1 rows)
```

Figura 30: Tercera consistencia 1.

Donde todas las operaciones se realizaron correctamente. Terminaremos repitiendo el proceso aplicando **CONSISTENCY QUORUM**.

```

cqlsh> CONSISTENCY QUORUM;
Consistency level set to QUORUM.
cqlsh> INSERT INTO actors.actors_by_id (actor_id, nombre, nacionalidad, fecha_nacimiento, edad, patrimonio, idiomas) VALUES (323e4567-e89b-12d3-a456-426614174003, 'Claire Dubois', 'Francia', '2001-07-09', 25, 2100000, {'Francés', 'Alemán', 'Español'});
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter 1>: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level QUORUM" info={\'consistency\': \'QUORUM\', \'required_replicas\': 2, \'alive_replicas\': 1\'})})
cqlsh> SELECT nombre FROM actors.actors_by_id WHERE actor_id = 323e4567-e89b-12d3-a456-426614174003;
NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter 1>: Unavailable('Error from server: code=1000 [Unavailable exception] message="Cannot achieve consistency level QUORUM" info={\'consistency\': \'QUORUM\', \'required_replicas\': 2, \'alive_replicas\': 1\'})})

```

Figura 31: Segunda consistencia Quorum.

Y como se puede observar, fallaron ambas operaciones. A modo de resumen se adjunta la siguiente tabla:

Nodos disponibles	ONE	QUORUM	ALL
3	Éxito	Éxito	Éxito
2	Éxito	Éxito	Falla
1	Éxito	Falla	Falla

Cuadro 5: Tabla resumen

6.2.1. Explicación de los resultados y cálculo de Quórum

Para calcular el Quórum se utiliza la fórmula:

$$\left\lceil \frac{RF}{2} \right\rceil + 1$$

Donde RF es el factor de replicación. Para este caso ($RF = 3$), el cálculo determina la mayoría estricta de réplicas requeridas:

$$\left\lceil \frac{3}{2} \right\rceil + 1 = \lceil 1.5 \rceil + 1 = 1 + 1 = 2$$

Esto explica por qué las consultas con CONSISTENCY QUORUM funcionan cuando hay 3 y 2 nodos disponibles, pero fallan cuando solo hay 1 nodo disponible.

Al utilizar CONSISTENCY ONE, el nodo coordinador solo necesita la confirmación de una réplica para dar la operación por válida. Por esta razón el sistema siguió respondiendo aún cuando ya se habían perdido dos nodos.

Al utilizar CONSISTENCY ALL, el coordinador necesita que los 3 nodos respondan. Si tan solo uno de estos nodos se cae, el sistema se vuelve no disponible para esa consulta, sacrificando la tolerancia a fallos para asegurar que se está leyendo la versión más reciente del dato.

Al utilizar CONSISTENCY QUORUM se llega a un balance, se garantiza una lectura fuertemente consistente mientras se tolera la pérdida de una minoría de los nodos de la topología.

6.3. Recuperación de nodos

Se volvieron a levantar los nodos y se ejecuto un SELECT sobre un dato ingresado en el test anterior, y como se puede apreciar en la imagen se mantuvo el cambio.

```

$ docker exec -it cassandra1 nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens  Owns (effective)  Host ID                               Rack
UN 172.26.0.3    155.93 KiB    64      100.0%            40494604-f0ba-4b5a-bc85-1ade051cb017 rack1
UN 172.26.0.2    153.05 KiB    64      100.0%            de7c8fb0-12d9-43a4-aa84-194bf43e5f80 rack1
UN 172.26.0.4    208 KiB       64      100.0%            a3259457-c1b4-4b85-afbf-8536344b4d45 rack1

$ docker exec -it cassandra1 cqlsh

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to MyCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.9 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> SELECT nombre FROM actors.actors_by_id WHERE actor_id = 323e4567-e89b-12d3-a456-426614174003;

  nombre
-----
  Claire Dubois

(1 rows)

```

Figura 32: Reinicio de nodos.

6.3.1. ¿Qué es el *hinted handoff*?

Cassandra replica cada escritura en varios nodos según el factor de replicación (en nuestro caso 3). Si al escribir una de las réplicas está caída o inaccesible, el *coordinador* (el nodo que recibió la petición del cliente) no puede entregarle la mutación en ese momento.

El *hinted handoff* es el mecanismo que evita perder esa escritura para esa réplica: el coordinador guarda localmente una *pista* (*hint*), que contiene la mutación y el identificador del nodo destino. Cuando detecta que el nodo volvió, mediante el protocolo *gossip*, le reenvía las pistas acumuladas y el nodo se pone al día sin intervención manual.

6.3.2. Escrituras dirigidas a una réplica temporalmente inaccesible

El flujo es el siguiente:

1. El cliente envía una escritura al coordinador.
2. El coordinador la envía a todas las réplicas. Una de ellas, R_3 , está caída.
3. Las réplicas disponibles la aplican. Si se alcanza el nivel de consistencia pedido (por ejemplo QUORUM, 2 de 3), el cliente recibe confirmación de éxito.
4. El coordinador almacena una pista para R_3 en disco.
5. Cuando R_3 vuelve, el coordinador le reenvía la pista y R_3 aplica la mutación.

Algunas características importantes:

- **Ventana limitada.** Las pistas solo se generan mientras el nodo lleve caído menos de `max_hint_window` (por defecto, 3 horas). Pasado ese tiempo el coordinador deja de acumularlas, para no llenar el disco.
- **No cuenta para la consistencia.** Una pista no equivale a una réplica escrita. Solo con el nivel ANY una pista basta para confirmar la escritura.

- **Es un mecanismo de mejor esfuerzo.** Si el propio coordinador falla antes de entregar las pistas, o si estas expiran, la réplica queda desactualizada.
- **Las pistas se reenvían de forma asíncrona**, con un límite de velocidad para no saturar al nodo recién recuperado.

6.3.3. Función del proceso de *repair*

El *hinted handoff* cubre solo caídas cortas. El *repair* (*anti-entropy repair*, `nodetool repair`) es el mecanismo que garantiza la convergencia de las réplicas a largo plazo. Funciona así:

- Cada réplica construye un árbol de Merkle (*hash tree*) de los datos de un rango de tokens.
- Los nodos comparan los árboles y detectan los rangos que difieren.
- Solo se transmiten los datos en desacuerdo, y se conserva en cada réplica la versión más reciente según las marcas de tiempo.

El *repair* corrige las inconsistencias que el *hinted handoff* no pudo evitar: nodos caídos más de `max_hint_window`, pistas perdidas o expiradas, y escrituras que nunca llegaron a alguna réplica. También es necesario para los borrados. Cassandra marca los datos eliminados con *tombstones*, que se descartan tras `gc_grace_seconds` (10 días por defecto). Si una réplica no recibió el borrado y el *tombstone* ya fue purgado en las demás, el dato eliminado puede reaparecer. Por eso se recomienda ejecutar el *repair* al menos una vez dentro de ese plazo.

	Hinted handoff	Repair
Cuándo actúa	Durante la escritura, ante un fallo	Periódicamente o por demanda
Alcance	Caídas cortas	Cualquier divergencia
Quién lo hace	El coordinador, automáticamente	Administrador, con <code>nodetool</code>
Garantía	Mejor esfuerzo	Convergencia completa

6.3.4. Relación entre replicación y tolerancia a fallos

La replicación es la base de la tolerancia a fallos en Cassandra: al guardar RF copias de cada dato en nodos distintos, el sistema sigue funcionando aunque algunos nodos fallen, sin un punto único de falla. Con $RF = 3$ y nivel QUORUM, que exige $\lfloor RF/2 \rfloor + 1 = 2$ respuestas, el clúster tolera la caída de una réplica por rango de tokens sin perder disponibilidad ni lecturas consistentes, ya que se cumple $R + W > RF$ (por ejemplo, $2 + 2 > 3$).

Estos mecanismos se complementan para lograr **consistencia eventual**:

- La **replicación** y el nivel de consistencia determinan cuántos fallos se toleran.
- El *hinted handoff* recupera rápidamente a una réplica tras una caída corta.
- El *read repair* corrige divergencias detectadas al leer.
- El *repair* completo restablece la consistencia entre todas las réplicas.

En términos del teorema CAP, esta combinación corresponde a un diseño orientado a disponibilidad y tolerancia a particiones (AP): permite seguir escribiendo con nodos caídos y confía en estos mecanismos para que las réplicas converjan después.

6.4. Actualización y datos desnormalizados

Se incluye un archivo llamado `evidencia.txt` en donde se realizan 3 consultas pre-actualización y 3 consultas post-actualización en donde se pueden ver reflejados los cambios que realizó la actualización.

Para llevar a cabo la actualización basta con ejecutar los siguientes comandos:

```
1 docker cp actualizacion.cql cassandra1:/tmp/actualizacion.cql
2
3 docker exec -it cassandra1 cqlsh -f /tmp/actualizacion.cql | tee evidencia.txt
```

Es importante notar que `tee evidencia.txt` será necesario para obtener el archivo de texto con las evidencias.

6.4.1. Por qué la desnormalización dificulta las actualizaciones

En Cassandra cada consulta tiene su tabla, así que un mismo dato vive en varias. Aquí *patrimonio* está en *actors_by_id* y en *actors_by_nationality*. Cassandra no tiene joins ni claves foráneas, por lo que la aplicación debe escribir en todas las copias. Esto introduce 3 problemas:

- Si una escritura falla o se omite, las copias quedan inconsistentes.
- Para actualizar *actors_by_nationality* hay que conocer nacionalidad, edad y actor_id. Primero se debe leer *actors_by_id* para obtenerlos, porque no se puede hacer UPDATE ... en esa tabla.
- Si un dato fuera parte de la clave no se podría modificar. Habría que borrar la fila e insertarla de nuevo.

6.4.2. Qué ventaja entrega

Cada consulta se resuelve leyendo una sola partición, sin joins ni filtros. Las lecturas son rápidas y escalan de forma predecible, a costa de escrituras más complejas y de más almacenamiento.

6.4.3. Uso de BATCH

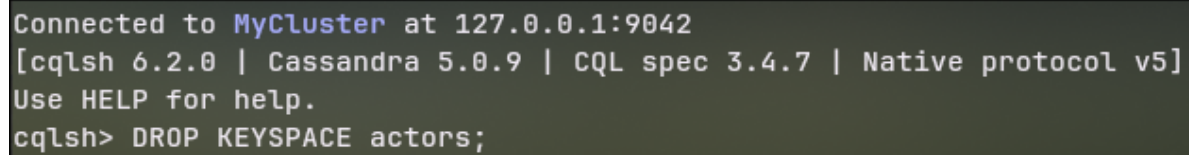
Si conviene usar BATCH para miles o millones de actualizaciones. Depende de qué se agrupe:

- **Sí, para agrupar las escrituras de un mismo actor.** Un BATCH LOGGED las vuelve atómicas, lo que mantiene consistentes las tablas desnormalizadas.
- **No, para agrupar miles o millones de actualizaciones distintas.** Un BATCH no acelera nada. Si abarca muchas particiones, el coordinador debe contactar a muchos nodos, y el batch log agrega escrituras extras. Los batches grandes sobrecargan al coordinador y pueden superar ciertos límites establecidos, así que pueden fallar.
- Para carga masiva conviene ejecutar muchas sentencias preparadas individuales y de forma asíncrona y concurrente, con un límite de concurrencia. Un UNLOGGED BATCH solo sirve cuando todas las sentencias van a la misma partición.

7. Parte 4: Cierre

Como todo lo que empieza tiene que acabar, se ejecuta el siguiente comando para eliminar el KEYSPACE y dejar todo como en su punto inicial.

```
1 DROP KEYSPACE actors;
```



```
Connected to MyCluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.9 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> DROP KEYSPACE actors;
```

Figura 33: DROP KEYSPACE.

8. Referencias y material complementario

- Apache Cassandra Documentation. *Data Modeling*.
- Apache Cassandra Documentation. *CQL: Data Manipulation (SELECT, ALLOW FILTERING)*.
- Apache Cassandra Documentation. *Storage-Attached Indexing (SAI)*.
- Apache Cassandra Documentation. *Tracing*.
- Enlace al repositorio público: <https://github.com/Sxbayb/Tareas-BDDA>.
- Material de clase. 2.3 Bases de Datos de Columnas Anchas.